Tell us about your PDF experience.

Azure Function App troubleshooting documentation

Welcome to Azure Function App troubleshooting. These articles explain how to determine, diagnose, and fix issues that you might encounter when you use Azure Function App. In the navigation pane on the left, browse through the article list or use the search box to find issues and solutions.

Application Code Deployment

HOW-TO GUIDE

Application issues post deployment

Availability

HOW-TO GUIDE

Function App Down or Reporting Errors

Configuring and Managing Function Apps

HOW-TO GUIDE

Migration of runtime/language version

Creating or Deleting Function Apps

HOW-TO GUIDE

General troubleshooting

Monitoring using Application Insights, Metrics and Alerts

HOW-TO GUIDE

Application Insights logs missing or incorrect

Resolve "Function App Down or Reporting Errors" issues

Common scenarios, error messages, and symptoms

Microsoft Azure Function App failures typically fit into three major categories: configuration issues, customer code issues, and platform issues. This article explores these issues in detail and introduces the appropriate diagnostic tools for troubleshooting.

Configuration issues

These issues are the most common. They include:

- Missing or incorrect application settings, such as `AzureWebJobsStorage`, `FUNCTIONS_WORKER_RUNTIME`, and binding-specific configurations. For more information, see the App settings reference for Azure Function App.
- Storage account misconfigurations, including deleted accounts, rotated keys, and firewall restrictions. For more information, see Storage considerations for Azure Function App.
- Key Vault and Managed Identity access problems because of misconfigured permissions or missing identity assignments. For more information, see the following resources:
  - Use Key Vault references as app settings - Azure App Service
  - Create a function app without default storage secrets in its definition - Azure Function App

Customer code issues

These issues have the following causes:

- Faulty code patterns that cause high CPU or memory usage, SNAT exhaustion, or runtime exceptions (for example, division by zero or null reference).
- Long-running or inefficient functions that are especially problematic on consumption plans because of timeout constraints.

For more information, see the following resources:

- Azure Function App best practices
- Improve Azure Function App performance and reliability
- Manage connections in Azure Function App

Platform issues

These issues are less common but usually have a greater effect:

- Unsupported runtime versions (for example, Azure Function App `~2.x` or `~3.x`). For more information, see the following resources:
  - Azure Function App language stack support policy
  - Supported languages in Azure Function App
- Host startup failures, including placeholder site specialization errors and container allocation issues.

Typical error messages

You might experience common errors such as the following:

- **Azure Function App runtime is unreachable.**
  - See Troubleshoot error: Azure Function App runtime is unreachable.
- **Function host not running.**
- **Access Denied**: `'C:\home\site\wwwroot\host.json'`.
- **HTTP 5*xx* errors**: 503, 502, 500, or timeouts.

Symptoms to watch for

- Function not triggering or running. See Analyze Azure Function App telemetry in Application Insights.
- Missing invocation history or unexpected gaps.
- High resource consumption (CPU or memory spikes that are flagged by detectors).
- Timeouts or long response times, especially on Consumption plans (maximum of 230 seconds).
- Functions not listed on the overview page.

Diagnostic tools

To identify and resolve issues, you can use the following tools in the Azure portal.

Function App Down or Reporting Errors (Preview)

To access this tool, follow these steps:

1. Sign in to the Azure portal.
2. Open **Function App**.
3. Select **Diagnose and solve problems**.
4. Search for **Function App Down or Reporting Errors (Preview)**.

Features of this tool

- Automates key troubleshooting checks by using existing detectors.
- Provides minimal noise and focused root cause analysis.
- Supports conversational diagnostics.

Issue categories

Function app downtime errors are categorized as follows.

Common configuration issues

- Misconfigured application settings
- Incorrect or unreachable storage account configuration
- Key Vault access permission issues
- Managed Identity authentication problems
- Binding or trigger misconfiguration
- Network connectivity issues

Code and runtime resource issues

- High CPU or memory consumption
- SNAT port exhaustion
- TCP connection exhaustion
- Long-running or stuck functions
- Runtime exceptions and failures
- Poor error handling or retry logic

Diagnostic checks performed

The diagnostic workflow evaluates the following aspects of your function spp:

- **General information**
  - Hosting plan type (Consumption, Premium, Dedicated, Flex)
  - Runtime version
  - Platform (Linux or Windows)

    - Trigger types and bindings
- **Startup issues**
    - Diagnostic events during app startup
    - Offline history analysis for unexpected downtimes
- **Recent deployments**
    - Highlights deployments that might have affected the app
- **Runtime and language version validation**
    - Confirms use of supported versions
- **Configuration checks**
    - Verifies mandatory app settings
    - Verifies Key Vault and Managed Identity configuration
    - Checks for SyncTrigger issues
    - Detects Function Host name collisions
- **Extension versions**
    - Identifies outdated or unsupported extensions
- **Hosting plan setup**
    - Analyzes configuration and scaling behavior
    - Checks for SNAT port exhaustion, high CPU, and memory issues
    - Includes risk alerts:
        - For Dedicated plans: verifies that **AlwaysOn** is enabled
        - For Elastic Premium: checks VNet routing and scaling
- **Execution health**
    - Detects:
        - Execution failures
        - Nontriggering functions
        - Stuck or long-running executions
- **AI-powered analysis**
    - Uses OpenAI to detect issue patterns and provide contextual recommendations

Application Insights queries

For diagnostics, run these queries in Application Insights:

- **Requests per worker (last 30 minutes):**

Kusto

```
requests
| where timestamp > ago(30m)
| summarize count() by cloud_RoleInstance, bin(timestamp, 1m)
| render timechart
```

- **Traces for errors (last 30 minutes):**

```
Kusto
traces
| where timestamp > ago(30m)
| where customDimensions.LogLevel == "Error"
```

- **Runtime exceptions (last 30 minutes):**

```
Kusto
exceptions
| where timestamp > ago(30m)
```

> **Note**
>
> Sampling is enabled by default. If it's necessary, disable this feature temporarily during an investigation.

For more information, see Analyze Azure Function App telemetry in Application Insights.

Network validator tool

> **Note**
>
> This tool applies to all hosting plans, except for the Windows and Linux Consumption plans.

If your app shows errors such as `FAILED TO INITIALIZE RUN FROM PACKAGE.txt` or `host.json not found`, use the network troubleshooter to resolve the errors.

To access the tool, follow these steps:

1. Sign in to the Azure portal.
2. Navigate to **Diagnose and solve problems**.
3. Search for **Connectivity Troubleshooter**.

This tool checks for:

- DNS resolution
- Storage and Key Vault access
- Outbound restrictions (VNet, NSG, firewalls)

When you use the network validator, make sure that:

- VNet integration is correctly configured.
- No endpoint or firewall blocks are in place.

For more information, see the following resources:

- Azure Function App networking options
- Frequently asked questions about networking in Azure Function App

Kudu logs (SCM)

> **Note**
>
> This tool applies to all hosting plans, except for the Linux Consumption and Flex Consumption plans.

For information about how to access and use Kudu logs (SCM), see Access Kudu for your app.

Other logs

- **Host Logs**: `%HOME%\LogFiles\Application\Functions\Host`
- **Functions log**: `%HOME%\LogFiles\Application\Functions\Function\<your_triggername>`
- **System-level log**: `%HOME%\LogFiles\Eventlog.xml`

For more information, see Understanding the Azure App Service file system.

Last updated on 10/31/2025

Resolve common runtime issues after deployment

This article describes the common causes of runtime issues and provides solutions to resolve these issues.

If your application is experiencing issues after deployment, it's important to determine whether the problem is related to deployment or runtime. A *deployment* issue can cause the wrong set of files to get deployed to your function app, or it can cause some files not to get deployed at all. A *runtime* issue occurs after deployment. The files in your **wwwroot** directory are exactly as they should be, but the function app isn't running correctly. When this scenario occurs, the technique that you used to deploy your site is no longer relevant. For a runtime issue, you should, instead, focus on what your code is doing at runtime and how it's failing.

Common causes for application runtime issues after content deployment

Runtime failure can occur for any of the following reasons:

- The function runtime can't start because the function app has lost access to the storage account.
- In the runtime sandbox environment of Azure Function App, something is blocking certain operations that work on your local computer.
- Your Azure Function App isn't configured correctly. For example, the function host doesn't start up because of incorrect values in the following settings:
  - Runtime or language version
  - Triggers connection strings
  - Key vault settings
- An external dependency (such as a database or messaging system) isn't set up correctly and is causing time-out errors.
- Sync triggers failed. This causes an incorrect loading of functions.
- Your code makes invalid assumptions about paths, such as hard-coding a path that exists only on your local computer.

Solutions

To fix runtime issues after the content deployment, try one or more of the following methods:

- Manually restart the function app.

- Run the Azure Function App Diagnostics to quickly diagnose and solve common function app problems.
- Verify whether storage connections are set up correctly and that the storage account is accessible.
- If you've enabled Application Insights for your function, see Application Insights logs to learn more about the full exception trace or error message that's causing errors.
- Review the function runtime migration guides if your deployment is updating the function app runtime or language version.

Contact us for help

If you have questions, you can ask Azure community support. You can also submit product feedback to Azure feedback community.

Last updated on 10/31/2025

Troubleshoot common issues during Azure Runtime upgrade

This article describes how to resolve some of the common issues that may occur during the update of the function app language or runtime version, like runtime being unreachable and modules not being found.

Actions to take if you receive an email "Action recommended: Update your Azure Function App to use .NET 6."

- Extended support for Microsoft .NET Core 3.1 ended on December 3, 2022. Azure Function App runtime v3 is based on .NET core 3.1. We recommend that you update your function app to runtime version 4.x, which uses .NET 6 and has long-term support. After December 3, 2022, your apps aren't eligible for new features, security patches, performance optimizations, or support until you upgrade them to Functions runtime version 4.x.

- Your Azure Function App on runtime v3 will continue to run, and your applications aren't impacted. You can deploy code to these Functions applications after this date. But we may remove the ability for you to create applications targeting the Functions runtime v3 using common paths.

- For more information and migration guidance, see:
  - Functions runtime versions
  - Migrate apps from Azure Function App version 3.x to version 4.x
  - Migrate apps from Azure Function App version 1.x to version 4.x

Validate function app's compatibility for runtime v4

- Navigate to your function app in the Azure portal. Select **Diagnose and solve problems** to open Azure Function App diagnostics. In the **Search** bar, type *Updating Function App Language or Runtime Version* to run it directly. The diagnostic report includes guidance on the update. After validation completes, follow the recommendations and address any issues in your app.
- We also provide a pre-upgrade validator to help you identify potential issues when migrating your function app to 4.x. From the same diagnostics search bar, type *Functions*

4.x Pre-Upgrade Validator to run it directly. After validation completes, follow the recommendations and address any issues in your app.

- We strongly recommend upgrading your local project environment to version 4.x. Fully test your app locally using version 4.x of the Azure Function App core tools.
- Consider using a staging slot to test and verify your app in Azure on the new runtime version before deploying to a production slot. Remember to set `WEBSITE_OVERRIDE_STICKY_EXTENSION_VERSIONS=0` for migration with slots.

Change the Functions runtime version for Windows

1. Set FUNCTIONS_EXTENSION_VERSION, which is an app setting in the Azure portal **Configuration** pane, to `~4`. For more information, see Change the runtime version.
2. Set the `netFrameworkVersion` site setting to target .NET 6. `netFrameworkVersion` is a SiteConfig setting, not an app setting. And it's not directly available in the Azure portal. But you can set it by using Azure Resource Explorer or Azure CLI/PowerShell.

For more information, see Functions runtime versions.

Change the Functions runtime version for Linux

1. Set FUNCTIONS_EXTENSION_VERSION, which is an app setting in the Azure portal **Configuration** pane, to `~4`. For more information, see Change the runtime version.
2. Set `LinuxFxVersion`, by using Command Line Interface (CLI), to `<Language>|<LanguageVersion>`, based on the language used. For more information, see Using LinuxFxVersion for Linux Function Apps. For example, to update to .NET 6, you can set `LinuxFxVersion` to `dotnet|6.0` and `FUNCTIONS_EXTENSION_VERSION` to `~4`.

For more information, see Functions runtime versions.

Get a list of supported languages by the runtime version

See the list of supported languages by the runtime version.

Update the language version setting for the function app after updating the code

- PowerShell: In the Azure portal, set the `PowerShell Core version`. For more information, see Change the PowerShell version.
- Python: Set `linuxFxVersion` to `python|3.x`. For more information, see Change the Python version.
- Java: Specify the language version by using `-DjavaVersion` as 11 or 17. For more information, see Specify the deployment version.
- Node, JavaScript: For Windows, set the `WEBSITE_NODE_DEFAULT_VERSION` app setting to `~16`. For Linux, set `linuxFxVersion`, by using CLI, to `node|16`. For more information, see Set the Node version.

Azure Function App Proxies usage

Support for proxies is available again in version 4.x so that you can successfully upgrade your function apps to the latest runtime version. However, we recommend that you switch to integrating your function apps with Azure API Management as soon as possible. API Management lets you take advantage of a more complete set of features to define, secure, manage, and monetize your Functions-based APIs. For more information, see How to migrate to APIM and Integrate Functions with APIM using Visual Studio.

Get a list of any instances of Azure Function App that use runtime version 1.x, 2.x, or 3.x

1. Navigate to your function app in the Azure portal.
2. Select **Diagnose and solve problems** to open Azure Function App diagnostics.
3. In the **Search** bar, type *Updating Function App Language or Runtime Version* to run it directly.
4. In the diagnostic report, scroll down and select **List All Function App** > **View Details**.
5. Set the search filters and select **Show** to get the list of functions apps in the current subscription.

Alternately, you can use App Service REST API calls (GetConfiguration) to determine this. `LinuxFxVersion` and `WindowsFxVersion` provide the version information.

Move a .NET Framework 4.6.1 app from Functions runtime v1 to v4

.NET Framework 4.6.1 has reached end-of-life, so apps should be upgraded to .NET Framework 4.8. In Functions v4, you can run a .NET framework 4.8 app in out-of-process mode. For more

information, see the isolated process guide.

A migrated function app isn't starting or has some runtime issues

- For .NET apps, remember to update the code to .NET 6 before building and deploying.
- Double-check the runtime version of the app in the Azure portal.
- For Windows apps, you must also set `netFrameworkVersion` by using CLI or PowerShell.
- Make sure you're using the 2.x or later extension bundles, as recommended. Version 4.x of the Functions runtime requires versions 2.x or 3.0.0.
- Logging in to Azure Storage using *AzureWebJobsDashboard* is no longer supported in 4.x. Instead, use Application Insights for monitoring.
- If you receive a message that Azure Function App runtime is unreachable and you're sharing storage accounts, see Host ID considerations.
- Azure Function App 4.x doesn't support Node.js 10 and 12, Python 3.6, and PowerShell 6. For more information, see Language versions supported for the v4 runtime.
- For Python functions, if there are errors related to module collision and module not found, try setting `PYTHON_ISOLATE_WORKER_DEPENDENCIES` to *1* in app settings.
- Review Migrating existing function apps.
- For runtime issues, from the **Diagnose and Solve Problems** option in the Azure portal, search for **Function app down or reporting errors** and review the diagnostic report for error messages and solutions. Also review the **Function Configuration checks** detector to make sure that the configuration is correct.
- Review the blog Issues when upgrading Azure function apps to V4 for additional troubleshooting tips.

Contact us for help

If you have questions or need help, create a support request, or ask Azure community support. You can also submit product feedback to Azure feedback community.

Last updated on 10/31/2025

Resolve issues when creating, deleting, or restoring Azure Function App

This article lists some common issues that may occur when you create, delete, or restore Azure Function App and steps to resolve the issues.

Create function apps

Azure Function App uses the Azure App Service infrastructure. You can create an Azure function app by using:

- Azure portal
- Azure CLI
- PowerShell
- Azure Resource Manager template or Bicep template

You can also create an app while deploying a function project from Visual Studio or Visual Studio Code.

Considerations for an Azure Storage account used by Azure Function App

When creating a function app, you can create or link to a general-purpose Azure Storage account that supports blob, queue, and table storage. Azure Function App relies on Azure Storage for operations, such as managing triggers and logging function executions. For more information, see Storage considerations for Azure Function App.

- Make sure that you have permissions to create a storage account and other resources. There can be no role-based access control (RBAC), policy, or scope violations.
- Use Managed Identity for Azure Storage account connection string, only for `AzureWebJobsStorage`. For more information, see Connecting to host storage with an identity (Preview).

Secure the Azure Storage account used by Azure Function App

You can create an Azure function app and a new storage account secured with private endpoints. Host the function app on a plan supporting virtual-network integration.

- For information on creation from the portal, see the networking tutorial.

- For Azure Resource Manager templates, see Azure/azure-quickstart-templates.
- You can also update an existing Functions app to point to a new secure storage account. For more information, see Restrict your storage account to a virtual network.

Migrate an Azure function app across different hosting plans

In Windows, you can use Azure CLI commands to migrate a function app between a Consumption and a Premium plan, or vice versa. For more information, see Plan migration.

- Direct migration from/to a dedicated (App Service) plan to/from a Consumption/Premium plan isn't supported because features for event-driven scaling and using serverless features differ across hosting plans.

> **Note**
>
> Linux doesn't support migrating hosting plans.

Review hosting plan (Consumption, Premium, Dedicated) offerings like virtual network connectivity, time-out, billing, and so on. For more information, see Azure Function App hosting options.

Resolve the error "This region has quota of 0 instances for your subscription. Try selecting different region or SKU"

If you're consistently experiencing resource creation issues in a particular region, try to create the resource in a geographically paired region if your business allows it. For example, *West US 2* and *West Central US* are paired regions. *East Asia (Hong Kong SAR)* and *Southeast Asia (Singapore)* are also paired regions.

For more information, see:

- Azure regions decision guide - Cloud Adoption Framework
- Cross-region replication in Azure

Resolve the error "The pricing tier is not allowed in this resource group" or "SkuNotAllowedForResourceGroup"

We recommend that you create the plan in a new resource group. Different SKUs require unique machines. When you create an app in a resource group, it's mapped and assigned to a pool of resources. If you create another plan in that resource group and the mapped pool lacks the required resources, the error appears.

For more information, see Creating function apps in an existing resource group.

Creation of a function app content file share fails with: "The remote server returned an error: (403) Forbidden"

For function apps in Consumption plan or Elastic Premium plan, a file share is created on the storage account and referenced by using the app setting `WEBSITE_CONTENT_SHARE`. This error may occur because the storage account has a firewall, private endpoints, or other virtual network security restrictions such as network security group (NSG) rules.

To resolve the error, create a file share in advance on the secure storage account, and configure it to `WEBSITE_CONTENT_SHARE`. For more information, see Restrict your storage account to a virtual network.

Alternative tools for creation

You can use alternative tools like Azure CLI, PowerShell, and `Az.Functions`. Make sure to use the latest versions of the tools. If there's an issue when using one of the tools, like Azure portal, try using another, like Azure CLI.

Set the function runtime version for Windows and Linux

For Windows apps, set the runtime version in the Azure portal (**Settings** > **Configuration** > **Function runtime Settings** > **Runtime version**), or create an application setting `FUNCTIONS_EXTENSION_VERSION` and set it to the major runtime version, for example, `~4` to target V4. For more information, see How to target Azure Function App runtime versions. Pin it to a minor version based on host releases by setting `FUNCTIONS_EXTENSION_VERSION` to a minor version. Also see Azure Function App host releases.

For Linux apps, see Manual version updates on Linux.

Troubleshoot HTTP 429 errors

Throttling happens at two levels: Azure Resource Manager and resource provider (Microsoft.Web). The level depends on the subscription type and the hosting plan. For more information, see:

- Azure Resource Manager throttling
- Azure subscription and service limits, quotas, and constraints
- Azure App Services limits

> **Note**
>
> If the issue is infrequent, wait and retry after some time to see if the issue resolves. If the issue is frequent, you can buy a different subscription or update the hosting plan to resolve the error.

Delete or restore function apps

After you publish code from Visual Studio and use `WEBSITE_RUN_FROM_PACKAGE`, the Azure portal sets functions as *read-only* to prevent editing precompiled assets in the portal. To delete functions in a function app, remove the unwanted functions from your code, enable the **Remove additional files at destination** option in profile settings, and redeploy your code.

Restoring Azure Function App hosted on a Consumption plan or Elastic Premium plan isn't supported. If you have the function app content or can access the storage account, update the app settings or create a new function app, and use the content. You can restore Azure Function App on a Dedicated App service plan if it isn't using Azure Files for content storage.

For more information, see Restore (undelete) deleted web apps.

Make sure that you have sufficient permissions to delete an Azure function app. There can be no role-based access control (RBAC), policy, or scope violations.

Collect data before contacting Microsoft support

If the function app creation or deletion fails, collect and share the following data when creating a support ticket:

- Azure portal: Reproduce the issue and capture a browser trace.
- Azure CLI or Azure PowerShell: Share the command that reported the error and verbose logging.
- For Azure Resource Manager deployment failures, find the error message in the Azure portal:
    1. Select **Resource Group** > **Deployments**.
    2. Share the failed task correlation ID.

Contact us for help

If you have questions or need help, create a support request, or ask Azure community support. You can also submit product feedback to Azure feedback community.

Last updated on 10/31/2025

Application Insights logs are missing or incorrect for Azure Functions apps

Article • 01/16/2025

You can closely monitor the function app through the integration between Azure Functions and Application Insights. And you can use Application Insights without any custom configuration.

If the Application Insights logs are missing, or if the data appears to be partial or inaccurate, use the following steps to resolve the issue.

Check configuration of function app

1. Navigate to your function app in the Azure portal.
2. Select **Diagnose and solve problems** to open Azure Functions diagnostics.
3. In the **Search** bar, type *Function Configuration Checks* and open it.
4. You see a diagnostic report of all function app configuration checks. In particular for Application Insights, the following checks are performed:
   - Only one of the following connection settings exists:
     - `APPINSIGHTS_INSTRUMENTATIONKEY` Application Insights Instrumentation key
     - `APPLICATIONINSIGHTS_CONNECTION_STRING` connection

       We recommend that you use the APPLICATIONINSIGHTS_CONNECTION_STRING for more stable behavior. The ability to use `APPINSIGHTS_INSTRUMENTATIONKEY` will be deprecated by 2025.
   - The `AzureWebJobsDashboard` built-in logging is disabled, as recommended.
   - Sampling is enabled for the Azure Functions telemetry (enabled by default).

Recommendation: The function app should be on version 4 and the runtime version should be at least 4.15.2*xx*. This is because, from this version onwards, you can track the log flows from Azure Functions to the Application Insights service. By monitoring the log flows, you can check for missing logs.

Custom application logs

By default, custom application logs you write are sent to the Functions host, which then sends them to Application Insights under the Worker category. However, some language stacks allow you to send the logs directly to Application Insights, which gives you full control over how logs you write are emitted. In this case, the logging pipeline changes from `worker > Functions host > Application Insights` to `worker > Application Insights`.

The following table summarizes the configuration options available for each stack:

Expand table

Language stack	Where to configure custom logs
.NET (in-process model)	`host.json`
.NET (isolated model)	Default (send custom logs to the Functions host): `host.json` To send logs directly to Application Insights, see Configure Application Insights in the HostBuilder.
Node.JS	`host.json`
Python	`host.json`
Java	Default (send custom logs to the Functions host): `host.json` To send logs directly to Application Insights, see Configure the Application Insights Java agent.
PowerShell	`host.json`

When you configure custom application logs to be sent directly, the host no longer emits them, and `host.json` no longer controls their behavior. Similarly, the options exposed by each stack apply only to custom logs, and they don't change the behavior of the other runtime logs described in this article. In this case, to control the behavior of all logs, you might need to make changes in both configurations.

Logs are missing or partial

Application Insights collects log, performance, and error data. Sampling configuration is used to reduce the volume of telemetry. The Sampling feature is enabled by default with the settings shown in the following host.json example. Excluded types aren't sampled.

JSON

```
{
  "logging": {
    "applicationInsights": {
      "samplingSettings": {
        "isEnabled": true,
        "maxTelemetryItemsPerSecond" : 20,
        "excludedTypes": "Request;Exception"
      }
    }
  }
}
```

If you notice any partially missing logs, this might occur because of sampling. To determine the actual sampling rate, use an Analytics query that uses the required time interval that's shown in the following code snippet. If you observe that the `TelemetrySavedPercentage` for any sampling type is less than 100, then that type of telemetry is being sampled.

Kusto

```
union requests,dependencies,pageViews,browserTimings,exceptions,traces
| where timestamp > todatetime("mm/dd/yyyy hh:mm:ss") and timestamp <
todatetime("mm/dd/yyyy hh:mm:ss")
| summarize TelemetrySavedPercentage = 100/avg(itemCount),
TelemetryDroppedPercentage = 100-100/avg(itemCount) by bin(timestamp, 1d),
itemType
| sort by timestamp asc
```

For more information, see Data collection, retention, and storage in Application Insights.

Control volume and verbosity of logs

You can increase or suppress the logs that are written. To do this, you can use a combination of log level and categories as configured in *host.json*.

The Azure Functions logger includes a category for every log. The category indicates which part of the runtime code or your function code generated the log. For example:

- The `Host.Results` and `Function.<YOUR_FUNCTION_NAME>` are some of the available categories.
- A log level is assigned to every log. The value indicates relative importance, such as `Warning` or `Information`.

For more information, see the other categories and log levels available.

You can configure how your application should write the logs by following the sample code snippet:

JSON

```
{
  "version": "2.0",
  "logging": {
    "logLevel": {
      "default": "Information", // catch all default, with modifications
below for individual categories.
      "Function": "Warning", // Warning level from all Functions (except the
ones configured below).
      "Host.Aggregator": "Trace", // Log all traces in the 'customMetrics'
table of (and shown on Metrics/Alerts blade in AI) - use either this or
Host.Results
      "Host.Results": "Error", // Error and Critical requests are only
logged in the 'requests' table of the AI (and shown on Monitor Functions
blade in Functions App) - use either this or Host.Aggregator
      "Function.Function1": "Information", //Information level logs from
Function 1, logged in 'traces', 'dependencies' and 'customMetrics' tables of
AI
      "Function.Function2.User": "Information" //user code logs from
Function2, logged in 'traces' table of AI
    },
    "applicationInsights": {
      "samplingSettings": {
        "isEnabled": true,
        "maxTelemetryItemsPerSecond": 1,
        "excludedTypes": "Exception"
      }
    }
  }
}
```

To configure these values at the App settings level (to avoid redeployment on *host.json* changes), override specific *host.json* values by creating an equivalent value as an application setting. For more information, see Override host.json values.

For more examples about how to suppress logs, see functions-log-suppression.

The virtual network integrated function app doesn't generate any logs

If a function app is integrated with a virtual network, you must open port 443 for outgoing traffic in your server firewall to allow the Application Insights SDK or Application Insights Agent to send data to the portal for the following URLs:

- *dc.applicationinsights.azure.com*
- *dc.applicationinsights.microsoft.com*
- *dc.services.visualstudio.com*
- **.in.applicationinsights.azure.com*

For more information, see IP addresses used by Azure Monitor.

Contact us for help

If you have questions or need help, create a support request, or ask Azure community support. You can also submit product feedback to Azure feedback community.

Feedback

Was this page helpful? Yes No